

AT32 BLDC Hall 1-Shunt Demo

Introduction

The purpose of the application note is to demonstrate how to drive a three-phase permanent magnet synchronous motor (BLDC motor) with Hall sensed six-step square wave control algorithms. The contents include environment settings, program document architecture, macro definitions and motor control state machine.

ArteryTek has many motor development boards. In this document, AT-MOTOR-EVB is used for demonstration.

AT-MOTOR-EVB Applicable products:

Part number	AT32F402/405 series
	AT32F413 series
	AT32F415 series
	AT32F421 series
	AT32F422/426 series
	AT32F425 series
	AT32L021 series
	AT32F45x series
	AT32M412/M416 series

Contents

1	Motor control library algorithms	5
2	Environment requirements	6
	2.1 Hardware requirements	6
	2.2 Software requirements	9
3	BLDC motor control library architecture.....	13
4	Motor control state machine.....	26
	4.1 Introduction.....	26
5	How to use demo project	30
6	Revision history.....	31

List of tables

Table 1. Motor related specifications	6
Table 2. Flash memory size and ROM configuration	9
Table 3. Motor control library file list	15
Table 4. Drive mode definitions	16
Table 5. Control parameter definition.....	16
Table 6. Driver parameters definition.....	18
Table 7. Motor parameter definition	19
Table 8. Configuration of peripherals (AT32F413RCT7)	22
Table 9. Configuration of peripherals (AT32F421C8T7).....	22
Table 10. Peripheral configuration functions	23
Table 11. Motor control interrupt functions.....	24
Table 12. Motor control library emulation list	24
Table 13. State switch conditions	28
Table 14. Document revision history.....	31

List of figures

Figure 1. JK42BLS01-X056ED motor.....	6
Figure 2. AT-MOTOR-EVB evaluation board.....	7
Figure 3. AT32F413RCT7 ROM configuration (Keil).....	10
Figure 4. AT32F413RCT7 ROM configuration (AT32IDE)	11
Figure 5. AT32F413RCT7 ROM configuration (IAR).....	12
Figure 6. Motor control program architecture	13
Figure 7. Structure of motor control files	13
Figure 8. Structure of motor control project.....	14
Figure 9. Relationship between BLDC BEMF, Hall state and MOS ON state (HALL_LEARN_DIR=0)	21
Figure 10. State machine flow chart.....	26

1 Motor control library algorithms

This document contains the following motor control algorithms:

Target motor type: Three-phase permanent magnet synchronous motor (BLDC)

Control modes:

- 120° square wave commutation

Three-phase PWM method:

- 120° conduction PWM control

Phase current sensing method:

- 1-shunt current sensing and reconstruction method

Rotor position detection mode:

- Hall-effect position sensor

Sensored 120° square wave control mode:

- 120° square wave voltage control
- Torque control (120° square wave current control)
- Speed control
- Forward and reverse control

Auto tuning functions:

- Hall status self-learning (for Hall sensor only)
- Winding parameters identification
- Current PI control parameters self-tuning

2 Environment requirements

2.1 Hardware requirements

A PMSM (BLDC) motor, AT-Link or third-party debugger, and motor control evaluation board (AT-MOTOR-EVB) are required.

For more information on hardware configurations, refer to *UM0011_AT32_LV_Motor_Control_EVB_User_Manual*.

- PMSM (BLDC) motor: JK42BLS01-X056ED

Figure 1. JK42BLS01-X056ED motor

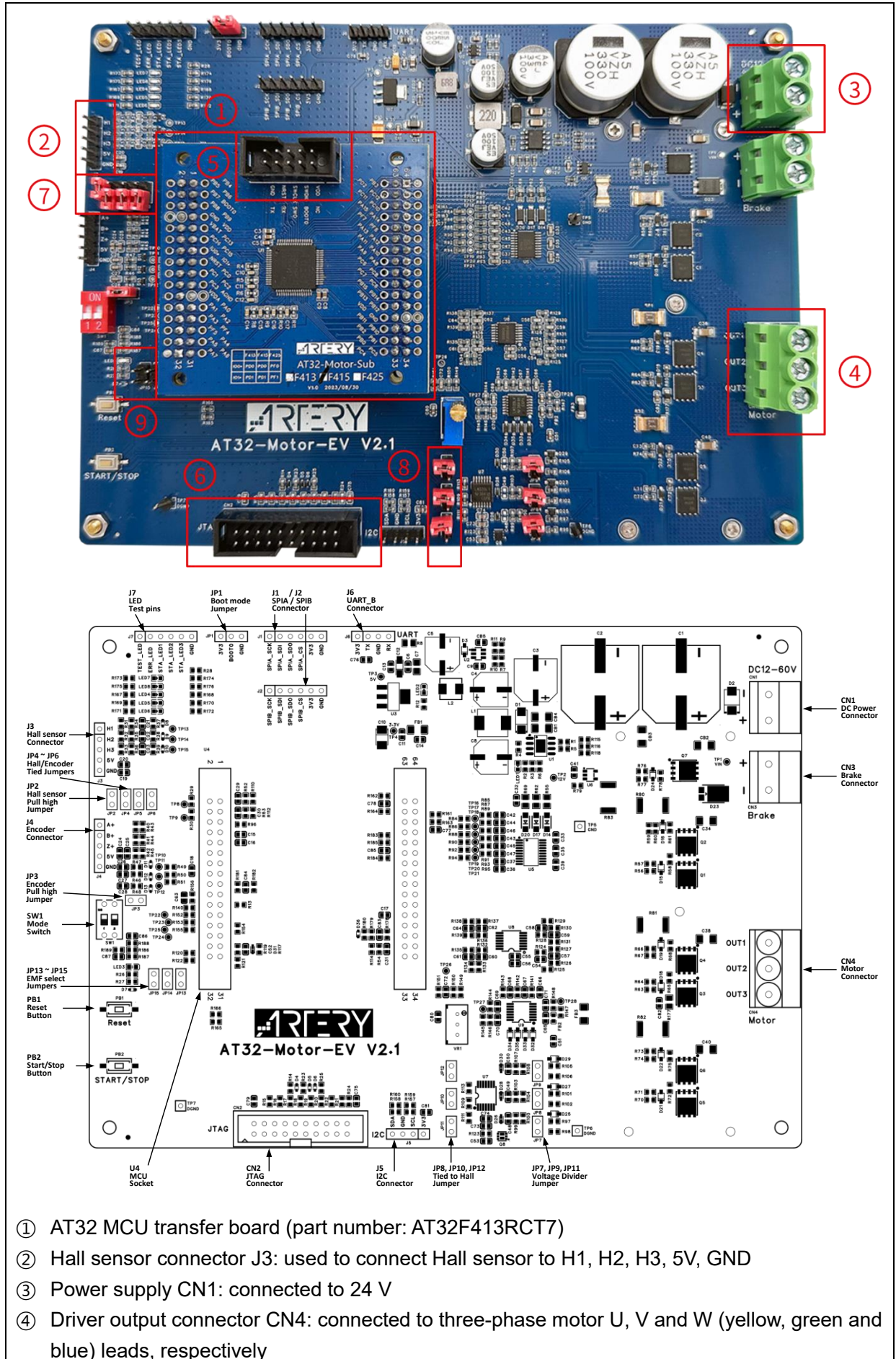


Table 1. Motor related specifications

Item	JK42BLS01-X056ED
Motor poles	8
Rated voltage	DC 24V
Rated speed	4000 RPM
Rated torque	0.0716 N·m
No-load current	0.3 Amps
Output power	30 W
Line-to-line resistance	1.8Ω
No-load speed	6800 RPM
BEMF constant	4.36 V/Krpm
Torque constant	0.042 N·m/A
Encoder resolution	1000 P/R
Insulation grade	Class B
Winding type	Delta

- Debugger
- Motor control evaluation board: AT-MOTOR-EVB (embeds an AT32F413RCT7 MCU)

Figure 2. AT-MOTOR-EVB evaluation board



- ⑤ AT-Link connector: choose this connector or J-Link connector
- ⑥ J-Link connector: choose this connector or AT-Link connector
- ⑦ Jumper settings: JP2 CLOSED; JP2, JP4, JP5, JP6 OPEN
- ⑧ Jumper settings: JP8, JP10, JP12 OPEN
- ⑨ Jumper settings: JP13, JP14, JP15 OPEN

2.2 Software requirements

- 1) Open “bldc_1shunt_hall_sensor” demo project.
- 2) Run “ArteryMotorMonitor.exe” (installation not required).
- 3) In Keil IDE, go to **Options - Read/Only Memory Areas** to set ROM according to the Flash memory size of each AT32 MCU, as shown in Table 2. For example, for AT32F413RCT7 with a 256 KB Flash memory, its IROM1 start address is 0x8000000 with a size of 0x3F800, whereas its IROM2 start address is 0x803F800 with a size of 0x800 (see Figure 3). In AT32IDE, modify *ld file according to AT32 MCU Flash memory size, as shown in Figure 4. In IAR Systems, modify *icf file according to the Flash memory size of each AT32 MCU, as shown in Figure 5.

Table 2. Flash memory size and ROM configuration

Flash size	4032K	1024K	512K	448K	256K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x3EF000	0xFF800	0x7F800	0x6F000	0x3F800
IROM2(address)	0x83EF000	0x80FF800	0x807F800	0x0806F000	0x803F800
IROM2(size)	0x1000	0x800	0x800	0x1000	0x800

Flash size	128K	64K	32K	16K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x1FC00	0x0FC00	0x07C00	0x03C00
IROM2(address)	0x801FC00	0x800FC00	0x8007C00	0x8003C00
IROM2(size)	0x400	0x400	0x400	0x400

Note [1]: Keil compiler version 5 is recommended since AT32 BSP source code does not support V6.15 compiler.

Figure 3. AT32F413RCT7 ROM configuration (Keil)

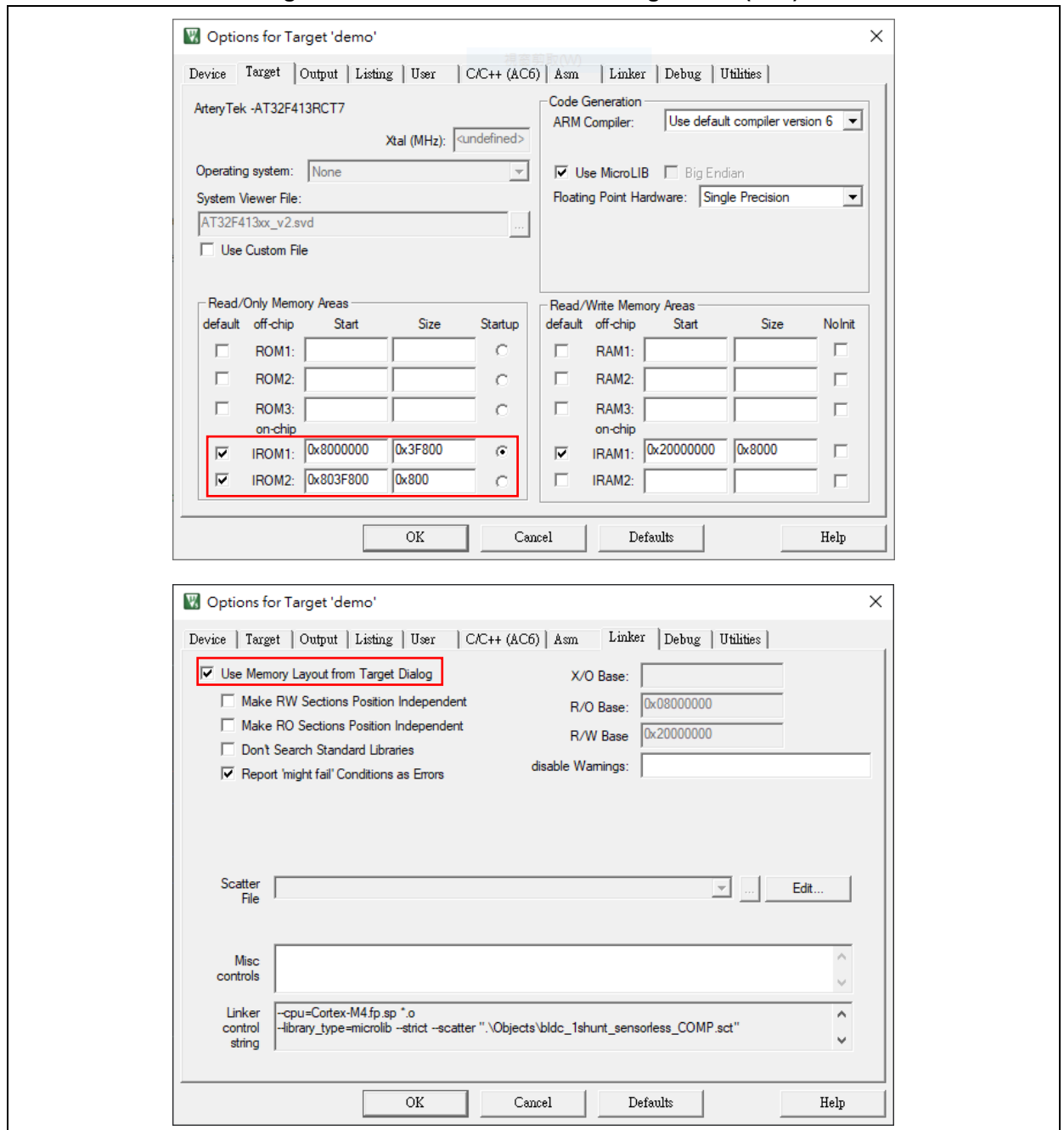


Figure 4. AT32F413RCT7 ROM configuration (AT32IDE)

```

AT32F413xC_FLASH.ld X
25 _estack = 0x20008000; /* end of RAM */
26
27 /* Generate a link error if heap and stack don't fit into RAM */
28 _Min_Heap_Size = 0x200; /* required amount of heap */
29 _Min_Stack_Size = 0x400; /* required amount of stack */
30
31 /* Specify the memory areas */
32 MEMORY
33 {
34     FLASH (rx)      : ORIGIN = 0x08000000, LENGTH = 0x3F800
35     MC_DATA (r)      : ORIGIN = 0x0803F800, LENGTH = 0x800
36     RAM (xrw)       : ORIGIN = 0x20000000, LENGTH = 32K
37 }
38
39 /* Define output sections */
40 SECTIONS
41 {
42     /* The startup code goes first into FLASH */
43     .isr_vector :
44     {
45         . = ALIGN(4);
46         KEEP(*(.isr_vector)) /* Startup code */
47         . = ALIGN(4);
48     } >FLASH
49
50     .mc_data :
51     {
52         . = ALIGN(4);
53         . = ORIGIN(MC_DATA);
54         _MC_VectStoreAddr1 = .;
55         *(.MC_VectStoreAddr1);
56         . = ALIGN(4);
57         . = ORIGIN(MC_DATA) + 800;
58         _MC_VectStoreAddr2 = .;
59         *(.MC_VectStoreAddr2);
60         . = ALIGN(4);
61     } >MC_DATA
62
63     /* The program code and other data goes into FLASH */
64     .text :
65     {
66         . = ALIGN(4);
67         *(.text)           /* .text sections (code) */
68         *(.text*)          /* .text* sections (code) */
69         *(.glue_7)          /* glue arm to thumb code */
70         *(.glue_7t)         /* glue thumb to arm code */
71         *(.eh_frame)
72

```

Figure 5. AT32F413RCT7 ROM configuration (IAR)

```

1  /****ICF**** Section handled by ICF editor, don't touch! *****/
2  /*-Editor annotation file-*/
3  /* IcfEditorFile="$TOOLKIT_DIR$\config\ide\IcfEditor\cortex_vl_0.xml" */
4  /*-Specials-*/
5  define symbol __ICFEDIT_intvec_start__ = 0x08000000;
6  /*-Memory Regions-*/
7  define symbol __ICFEDIT_region_ROM_start__ = 0x08000000;
8  define symbol __ICFEDIT_region_ROM_end__ = 0x0803F800 - 1;
9  define symbol __ICFEDIT_region_ROM1_start__ = 0x0803F800;
10 define symbol __ICFEDIT_region_ROM1_end__ = (__ICFEDIT_region_ROM1_start__ + 800 - 1);
11 define symbol __ICFEDIT_region_ROM2_start__ = (__ICFEDIT_region_ROM1_end__ + 1);
12 define symbol __ICFEDIT_region_ROM2_end__ = 0x0803F800 + 0x800 - 1;
13 define symbol __ICFEDIT_region_RAM_start__ = 0x20000000;
14 define symbol __ICFEDIT_region_RAM_end__ = 0x20007FFF;
15 /*-Sizes-*/
16 define symbol __ICFEDIT_size_cstack__ = 0x400;
17 define symbol __ICFEDIT_size_heap__ = 0x200;
18 /**** End of ICF editor section. ****ICF*****/
19
20 define memory mem with size = 4G;
21 define region ROM_region = mem:[from __ICFEDIT_region_ROM_start__ to __ICFEDIT_region_ROM_end__];
22 define region ROM1_region = mem:[from __ICFEDIT_region_ROM1_start__ to __ICFEDIT_region_ROM1_end__];
23 define region ROM2_region = mem:[from __ICFEDIT_region_ROM2_start__ to __ICFEDIT_region_ROM2_end__];
24 define region RAM_region = mem:[from __ICFEDIT_region_RAM_start__ to __ICFEDIT_region_RAM_end__];
25
26 define block CSTACK with alignment = 8, size = __ICFEDIT_size_cstack__ { };
27 define block HEAP with alignment = 8, size = __ICFEDIT_size_heap__ { };
28
29 initialize by copy { readwrite };
30 do not initialize { section .noinit };
31
32 place at address mem:__ICFEDIT_intvec_start__ { readonly section .intvec };
33
34
35 place in ROM_region { readonly };
36 place in ROM1_region { readonly section .MC_VectStoreAddr1 };
37 place in ROM2_region { readonly section .MC_VectStoreAddr2 };
38 place in RAM_region { readwrite,
39 block CSTACK, block HEAP };

```

3 BLDC motor control library architecture

Figure 6 presents the relationship between motor control functions, BSP, UI programs and hardware peripherals. In this motor control library architecture, the underlying hardware peripherals are controlled with BSP functions. The motor control functions and UI functions are built based on BSP and low-level functions. The user-defined programs are based on the motor control functions and UI functions. This organization makes it easier for users to call motor control functions to control MCU hardware peripherals and execute motor control programs. At the same time, it is also possible to obtain real-time motor control status or adjust motor control parameters by using UI control functions linked to the external PC UI software.

Figure 6. Motor control program architecture

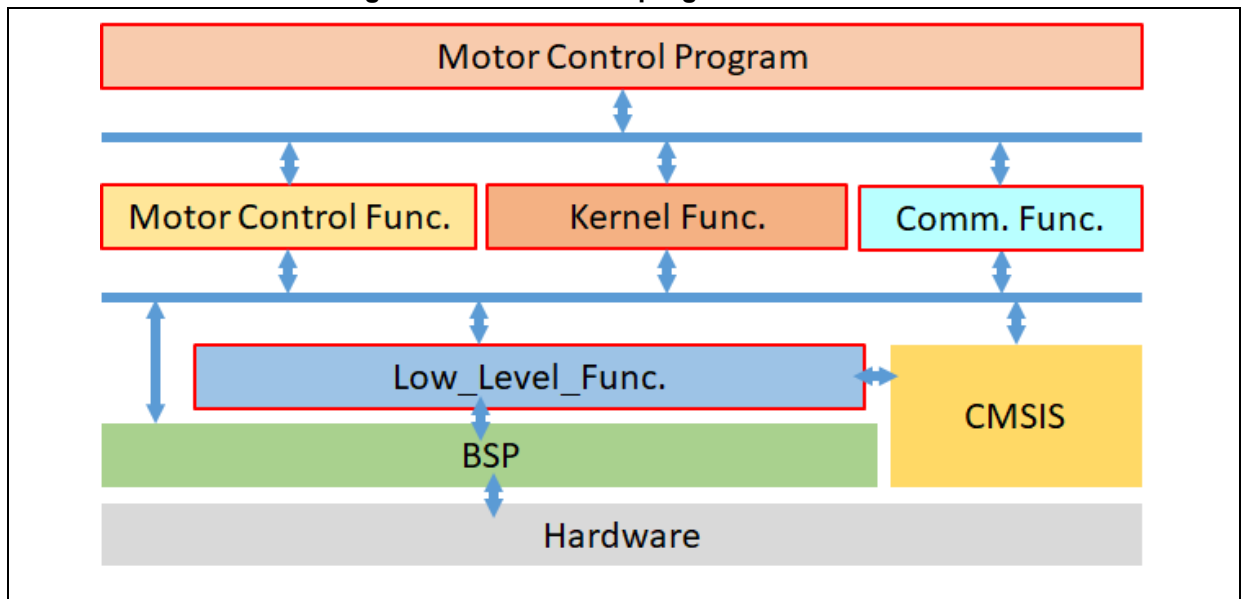


Figure 7 illustrates the overall structure of motor control files. The “motor_control_drive_param.h” header file is used to configure drive mode, control parameters, motor parameters and board parameters, while the “mc_hwio.h” header file is used to configure MCU peripheral parameters. All of these parameters are then integrated into the “mc_foc_globals.h (mc_bldc_globals.h)” and programmed with the initial variables through the “mc_foc_globals.c (mc_bldc_globals.c)”. The “mc_hwio.c” can be used to initialize MCU peripherals.

Figure 7. Structure of motor control files

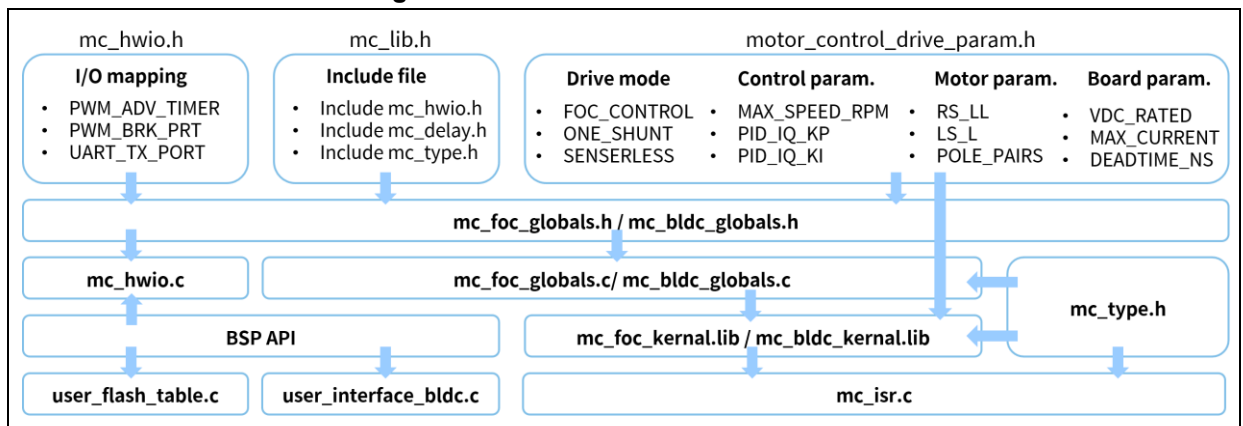


Figure 8 demonstrates the organization of files in the bldc_1shunt_hall_sensor demo. The “user” is a user-defined folder containing the main programs, peripheral programs and parameter definition header files. The “firmware” folder holds BSP files. The “mclib” folder is used to contain motor control library programs, including delay functions, communication functions, global variable setting functions and motor control library functions.

For details, refer to *AN0064 AT32_Motor_Control_Library_User_Guide* on Artery official website.

Figure 8. Structure of motor control project

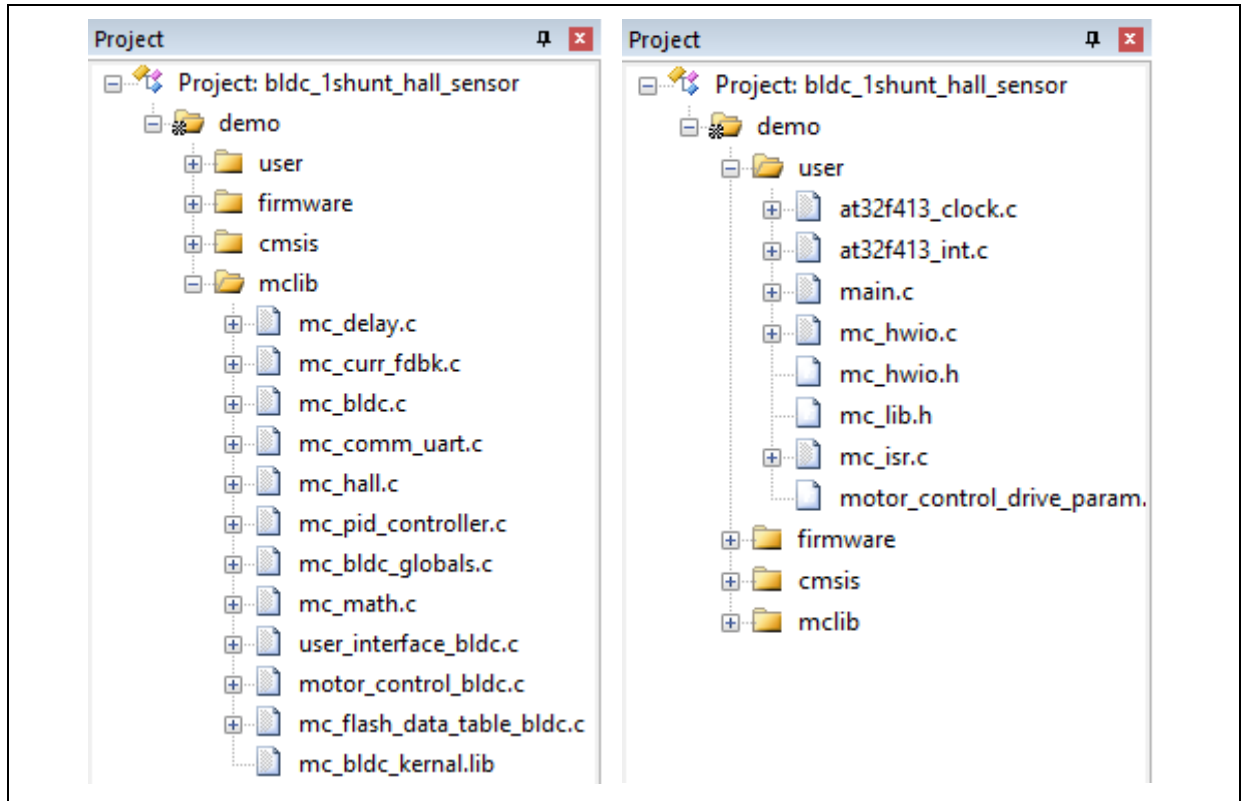


Table 3 gives a list of motor control library files.

Table 3. Motor control library file list

File name	Description
mc_lib.h	Header files
motor_control_drive_param.h	Define motor drive mode (current sampling mode, sensed mode, etc.), control parameters, motor parameters, board parameters
mc_hwio.c	Hardware peripheral configuration
mc_hwio.h	Hardware IO macro definitions
mc_isr.c	Motor control interrupt functions
mc_type.h	Global variables type and definition, enumeration definition
mc_delay.c	Delay functions
mc_delay.h	Declaration of delay functions
mc_comm_uart.c	Communication peripheral configuration
mc_comm_uart.h	Declaration and configuration of UART functions
mc_pid_control.c	PID controller functions
mc_pid_control.h	Declaration of PID controller functions
mc_curr_fdbk.c	Current sensing functions
mc_curr_fdbk.h	Declaration of current sensing functions
mc_math.c	Filter functions
mc_math.h	Declaration of filter functions
mc_hall.c	Hall sensor functions
mc_hall.h	Declaration of Hall sensor functions
motor_control_bldc.c	Motor control functions
motor_control_bldc.h	Declaration of motor control functions
mc_bldc.c	6-step square wave functions
mc_bldc.h	Declaration of 6-step square wave functions
mc_bldc_globals.c	Global variables definition and default value, global function definition
mc_bldc_globals.h	Global variables, global function declaration, macro definitions
user_interface_bldc.c	Communication interface functions
user_interface_bldc.h	Declaration of communication interface functions
mc_flash_data_table_bldc.c	Write Flash data table
mc_flash_data_table_bldc.h	Write Flash data table configuration

1) **motor_control_drive_param.h** file

- This file is divided into four parts and used to configure drive mode, control parameters, motor parameters and board parameters.
- Table 4 is a list of drive mode macro definitions. The user can use them to define the desired mode. This demo project uses the following features: six-step square wave control mode, 1-shunt current sensing, Hall sensor, low-speed voltage control mode, winding parameters identification and Artery motor UI tool feature.

Table 4. Drive mode definitions

Definition	Description
INTERNAL_CLOCK_SOURCE	Use an internal crystal (disabled by default)
AT_MOTOR_EVB_V1	V1.x motor control evaluation board
AT_MOTOR_EVB_V2	V2.x motor control evaluation board
COMPLEMENT	Enable complementarity between low-side MOS transistor switch and upper-side PWM (enabled by default, comment to disable)
GATE_DRIVER_LOW_SIDE_INVERT	Enable low-side MOS inverted output (enabled by default for AT32-Motor-EV, disabled by default for AT32M412-LV-Motor-EV)
HALL_SENSORS	Hall sensor
WITHOUT_CURRENT_CTRL	Pure voltage control, without current loop (disabled by default)
LOW_SPEED_VOLT_CTRL	Low speed voltage control mode (enabled by default, comment to disable)
MOTOR_PARAM_IDENTIFY	Motor winding parameters identification
USE_MOTOR_MONITOR	Artery motor UI tool (enabled by default, comment to disable)

- Table 5 presents the motor control parameters that are used in this demo. The user can set the desired parameters according to actual needs, such as Is current PI controller, speed PI controller, etc.

Table 5. Control parameter definition

Definition	Description
PWM_FREQ	PWM output frequency (unit: Hz)
MOTOR_CONTROL_MODE	Motor control mode (speed control, torque control, etc.; see motor_control_mode in the <i>type.h</i> for details)
CTRL_SOURCE	Control source settings (external source control/software control)
UI_UART_BAUDRATE	Baud rate of UI communication serial port (preset value: 1.5M)
UI_SAMPLE_CYCLE	Sampling frequency for motor tool parameters (unit: PWM counter timing base)

Definition	Description
TUNE_TARGET_CURRENT	Target current value for tuning current loop PI control parameters (unit: ampere)
TUNE_CURRENT_TOTAL_PERIOD	Total period for tuning current pulse for current loop PI control parameters (unit: ms)
TUNE_CURRENT_STEP_PERIOD	Step period for tuning current pulse for current loop PI control parameters (unit: ms)
I_SAMP_MIN_TIME	Minimum time for sampling ADC values (unit: nsec)
I_SAMP_DELAY	Drive signal delay time plus current sensing signal delay time (unit: nsec)
SPEED_FILTER_TIMES	Speed moving average times (unit: times)
MIN_SPEED_RPM	Minimum motor speed (unit: rpm)
MAX_SPEED_RPM	Maximum motor speed (unit: rpm)
ACC_SPD_SLOPE	Slope of acceleration (unit: rpm/ms when speed control loop=1kHz)
DEC_SPD_SLOPE	Slope of deceleration (unit: rpm/ms when speed control loop=1kHz)
MIN_SENSE_SPEED	Minimum speed of motor speed control loop (unit: rpm)
SP_MAX_VOLT	Maximum voltage of external command source (unit: voltage)
SP_THRESHOLD	Threshold voltage of external command source (unit: voltage)
SP_RUN_VALUE	Lowest voltage to startup in external source mode (unit: voltage)
SP_STOP_VALUE	Threshold voltage to stop in external source mode (unit: voltage)
OLC_INIT_SPD	Open loop initial speed (unit: rpm)
OLC_FINAL_SPD	Open loop final speed (unit: rpm)
OLC_TIMES	Set the times from initial speed to rise up to final speed (unit: times)
OLC_INIT_VOLT	Open loop initial voltage (unit: V)
OLC_VOLT_INC	Open loop incremental voltage (unit: V)
LEARN_TIME	HALL self-learning time (unit: msec)
LEARN_ALIGN_TIME	HALL self-learning align time (unit: msec)
PID_SPD_KP_DIV	Speed control proportional gain divisor (Q16 mode)
PID_SPD_KI_DIV	Speed control integral gain divisor (Q16 mode)
PID_SPD_KP_DEFAULT	Speed control proportional gain (Q15 mode)
PID_SPD_KI_DEFAULT	Speed control integral gain (Q15 mode)
PID_IS_KP_DIV	Bus current control proportional gain divisor (Q16 mode)
PID_IS_KI_DIV	Bus current control integral gain divisor (Q16 mode)
PID_IS_KP_DEFAULT	Bus current control proportional gain (Q15 mode)

Definition	Description
PID_IS_KI_DEFAULT	Bus current control integral gain (Q15 mode)
PID_SPD_VOLT_KP_DEFAULT	Low-speed voltage control proportional gain (Q15 mode) (for LOW_SPEED_VOLT_CTRL and WITHOUT_CURRENT_CTRL only)
PID_SPD_VOLT_KI_DEFAULT	Low-speed voltage control integral gain (Q15 mode) (for LOW_SPEED_VOLT_CTRL and WITHOUT_CURRENT_CTRL only)
PID_SPD_VOLT_KP_GAIN_DIV	Low-speed voltage control proportional gain divisor (Q16 mode) (for LOW_SPEED_VOLT_CTRL and WITHOUT_CURRENT_CTRL only)
PID_SPD_VOLT_KI_GAIN_DIV	Low-speed voltage control integral gain divisor (Q16 mode) (for LOW_SPEED_VOLT_CTRL and WITHOUT_CURRENT_CTRL only)
HYSTERESIS_LOW_SPEED	Minimum speed from current controller to voltage controller in low-speed voltage mode (unit: rpm) (for LOW_SPEED_VOLT_CTRL only)
HYSTERESIS_HIGH_SPEED	Maximum speed from voltage controller to current controller in low-speed voltage mode (unit: rpm) (for LOW_SPEED_VOLT_CTRL only)

- Table 6 presents the driver parameters that are used in this demo, including dead time, shunt resistance and current amplifier gain. Note that the corresponding parameters should be modified if a different motor drive board is to be used.

Table 6. Driver parameters definition

Definition	Description
VDC_RATED	DC-BUS voltage
BAT_LOW_VOLT	Lowest battery voltage (unit: voltage)
V_SENSE_GAIN	Voltage feedback ratio (refer to the corresponding motor control EVB user manual)
MAX_CURRENT	Maximum peak phase current of motor (unit: ampere)
MIN_CURRENT	Minimum peak phase current of motor (unit: ampere)
CURRENT_SPAN_SHIFT	Number of shifts for current per-unit normalization
ADC_REFERENCE_VOLT	ADC reference voltage (unit: voltage)
ADC_DIGITAL_SCALE_12BITS	ADC resolution
SYSTEM_CORE_CLOCK	System frequency (unit: Hz)
TMR_CLK	Timer clock frequency (unit: Hz)
DEADTIME_CLK_SFT_BITS	Shift bit number of frequency divider for dead time generator
DEADTIME_NS	Dead time (unit: ns)

Definition	Description
R_SHUNT	Shunt resistance (unit: Ω)
OP_GAIN	Current amplifier gain (refer to the corresponding motor control EVB user manual)
CURR_OFFSET_VOLT	Zero current offset (refer to the corresponding motor control EVB user manual) (unit: voltage)
OVER_CURRENT_SW	Software overcurrent threshold point (refer to the corresponding motor control EVB user manual) (unit: ampere)
DAC_VREF_SOURCE	Internal OP overcurrent protection DAC source selection (for AT32 MCUs with internal OPs, such as AT32M412)
OCP_CURRENT	Hardware overcurrent threshold point (unit: ampere)
BUS_CURR_CMP_OCP_VOLT	Internal OP overcurrent protection voltage (for AT32 MCUs with internal OPs, such as AT32M412) (unit: voltage)
TMR_BRK_FILTER_COUNT	Overcurrent protection break input filtering times (for AT32 MCUs with break input, such as AT32M412)
OVER_VOLT_THRESHOLD	Over-voltage threshold point (unit: voltage)
UNDER_VOLT_THRESHOLD	Under-voltage threshold point (unit: voltage)
V0_V	Parameter V0 of the approximate curve of NTC temperature vs. voltage (Note [2])
T0_C	Parameter T0 of the approximate curve of NTC temperature vs. voltage (Note [2])
dV_dT	Parameter dV/dT of the approximate curve of NTC temperature vs. voltage (Note [2])
OVER_TEMP_THRESHOLD	Over-temperature threshold point (unit: Celsius degrees)
MC_ERROR_MASK	Error detection mask

Note [2]: Approximate curve equation of voltage vs. temperature is $V[V]=V0+dV/dT[V/Celsius]*(T-T0)[Celsius]$.

- Table 7 presents the motor parameters used in this demo, including number of pole-pairs, encoder parameters and Hall sensor parameters.

Table 7. Motor parameter definition

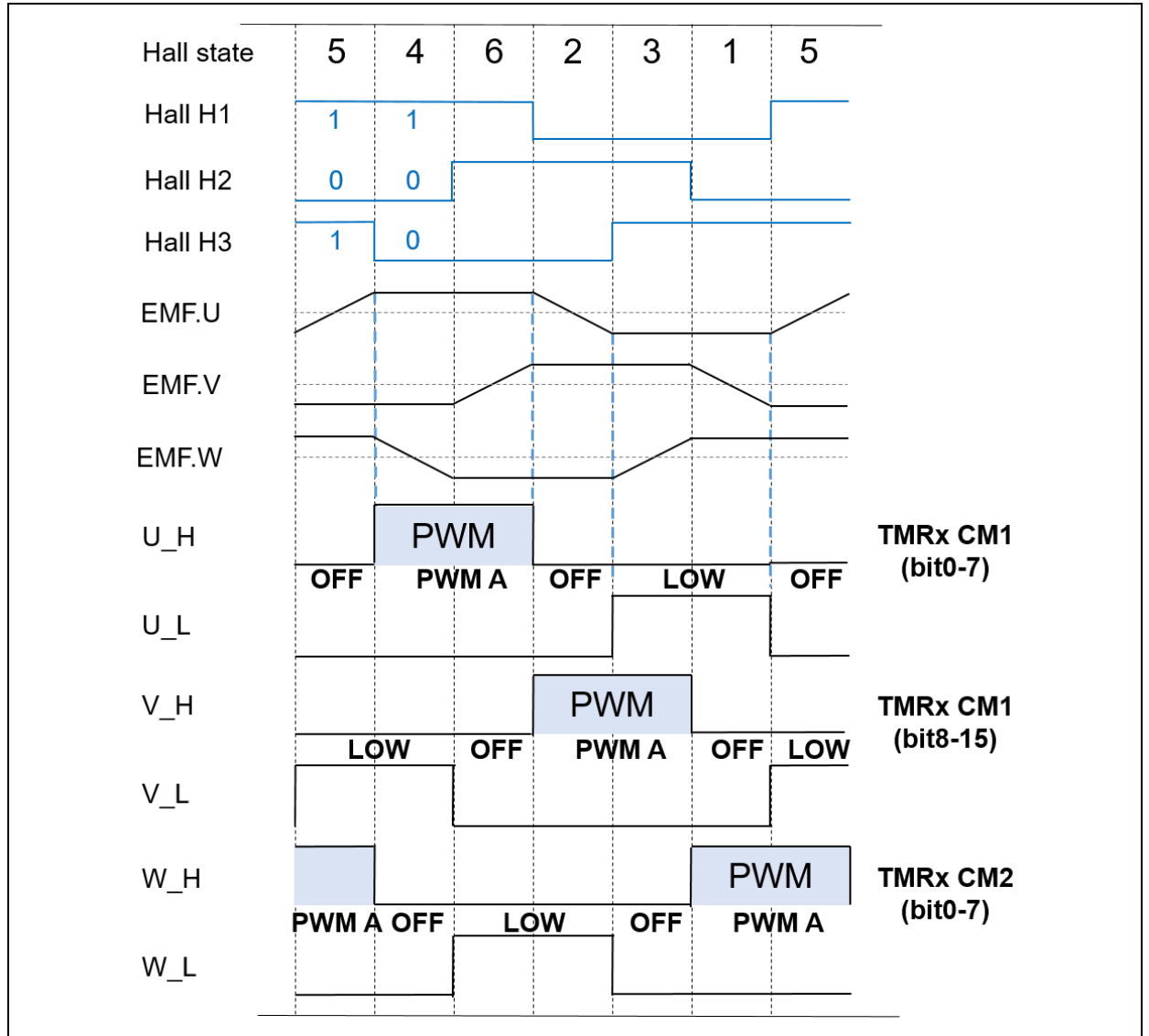
Definition	Description
RS_LL	Line-to-line winding resistance (unit: Ω)
LS_LL	Line-to-line winding inductance (unit: H)
POLE_PAIRS	Number of pole-pairs
KE	KE value (unit: V/rpm)
NOMINAL_CURRENT	Nominal current (unit: ampere)
HALL_LEARN_DIR	Motor direction after Hall sensor self-learning
HALL_LEARN_0_STATE	State 0 after Hall sensor self-learning (BLDC: output V-phase upper-side PWM and W-phase lower-side ON)

Definition	Description
HALL_LEARN_1_STATE	State 1 after Hall sensor self-learning (BLDC: output V-phase upper-side PWM and U-phase lower-side ON)
HALL_LEARN_2_STATE	State 2 after Hall sensor self-learning (BLDC: output W-phase upper-side PWM and U-phase lower-side ON)
HALL_LEARN_3_STATE	State 3 after Hall sensor self-learning (BLDC: output W-phase upper-side PWM and U-phase lower-side ON)
HALL_LEARN_4_STATE	State 4 after Hall sensor self-learning (BLDC: output U-phase upper-side PWM and V-phase lower-side ON)

Figure 9 illustrates the relationship between BLDC BEMF, Hall state and MOS ON when HALL_LEARN_DIR=0. For example, we can see from Figure 8 that Motor state 2 corresponds to the maximum BEMF for V-phase and the minimum BEMF for W-phase, and so the HALL_LEARN_0_STATE is set to 2, and so on and so forth. However, there is no need for AT32 MCU users to spend time finding this relationship as AT32 motor control library comes with a Hall phase sequence self-learning feature capable of providing corresponding Hall state values. This is achieved by writing Hall state values after Hall phase sequence self-learning routine into the corresponding hall sensor macro definitions. See Section 5.2 for details.

The following PWM output state refers to the scenario when using a gate driver with non-inverted lower-side input signals, and disabling lower-side PWM complementarity upon upper-side PWM toggle. If the user selects a gate driver with inverted lower-side input and enables PWM complementarity output upon upper-side PWM toggle, it is possible to change output mode with the motor_control_drive_param.h function, for example, our three six-step square wave control demo projects use the gate driver with inverted lower-side output and PWM complementary output enabled.

Figure 9. Relationship between BLDC BEMF, Hall state and MOS ON state (HALL_LEARN_DIR=0)



Note [3]: MOS upper-side and lower-side ON/OFF states are configurable according to BEMF and Hall state.

2) mc_hwio.h file

- This file is used to configure hardware IOs and peripherals. It also contains the declaration of the *mc_hwio.c* files. Table 8 and Table 9 present the configuration of peripherals (ADC, TMR, USART and EXINT, etc.), interrupt functions and DMA channels that are used in this demo.

Table 8. Configuration of peripherals (AT32F413RCT7)

Peripheral	Interrupt function	DMA channel	Description
ADC1	N/A	DMA1_CH1	ADC1 ordinary channel conversion (BUS voltage, MOS temperature, external speed/torque command)
ADC1	ADC1_2_IRQn	N/A	ADC1 preemptive channel conversion (bus current)
TMR1	TMR1_OVF_TMR10_IRQn	N/A	BLDC control interrupt
TMR1	TMR1_BRK_TMR9_IRQn	N/A	PWM output disable
TMR3	TMR3_GLOBAL_IRQn	N/A	Capture Hall sensor signal
USART1_TX	N/A	DMA1_CH2	USART1 TX data
USART1_RX	USART1_IRQn	DMA1_CH3	USART1 RX data
EXINT	EXINT15_10_IRQn	N/A	START/STOP button (start/stop motor)

Table 9. Configuration of peripherals (AT32F421C8T7)

Peripheral	Interrupt function	DMA channel	Description
ADC1	N/A	DMA1_CH1	ADC1 ordinary channel conversion (BUS voltage, MOS temperature, external speed/torque command)
ADC1	ADC1_CMP_IRQn	N/A	ADC1 preemptive channel conversion (bus current)
TMR1	TMR1_BRK_OVF_TRG_HALL_IRQn	N/A	BLDC control interrupt, PWM output disable
TMR3	TMR3_GLOBAL_IRQn	N/A	Capture Hall sensor signal
USART1_TX	N/A	DMA1_CH2	USART1 TX data
USART1_RX	USART1_IRQn	DMA1_CH3	USART1 RX data
EXINT	EXINT15_4_IRQn	N/A	START/STOP button (start/stop motor)

3) mc_hwio.c file

- This file configuration depends on users' hardware peripherals, such as TMR, ADC, DMA and GPIO. It also contains functions such as button, LED, PWM ON/OFF. See Table 10 for details.

Table 10. Peripheral configuration functions

Function	Description
nvic_config	Interrupt priority configuration
tmr_pwm_init	PWM timer, crm clock, GPIO and DMA configuration
gpio_hall_init	Hall sensor GPIO configuration
tmr_hall_init	Hall sensor timer and crm clock configuration
adc_ordinary_config	ADC ordinary channel ADC, DMA and GPIO configuration
adc_preempt_config	ADC preemptive channel ADC, DMA and GPIO configuration
uart_init	UART crm clock, GPIO and UART configuration
button_switch_init	Dip switch configuration
button_exint_init	Button interrupt event EXINT configuration
led_config	LED GPIO initialization and crm clock configuration
led_on	LED ON
led_off	LED OFF
led_toggle	LED toggles (from ON to OFF, or from OFF to ON)
led_blink	LED blinks
led_init	LED initialization configuration
current_offset_tmr_setting	Timer configuration for current offset calibration
get_int_vref_cal_ratio	Detect VDDA reference voltage ratio, used for ADC value calibration
motor_parameter_ID_config	Set TMR and ADC relating to motor winding parameter identification
hwdiv_config	Hardware divider configuration (for AT32L021 series only)
opa_init	Bus current OP configuration (for AT32 MCUs with internal OPs, such as AT32M412)
opa_calibration	Bus current OP calibration (for AT32 MCUs with internal OPs, such as AT32M412)
ocp_cmp_config	Bus current protection CMP configuration (for AT32 MCUs with internal CMPs, such as AT32M412)

4) mc_isr.c file

- This file contains interrupt handler functions for ADC, TMR and SYSTICK interrupts, etc. Refer to Table 11 for details.

Table 11. Motor control interrupt functions

Function	Description
ADVTMR_PWM_CYCLE_IRQ	TIMER1 update interrupt function: voltage loop control, current loop control, current sampling time counting, UI data transfer
ADVTMR_PWM_BRK_IRQ	TMR1 brake input interrupt function: PWM output disable
ADC_SHUNT_SAMP_READY_IRQ	ADC1 interrupt function: current sampling
HALL_CAPTURE_IRQ	TMR3 capture interrupt function: Hall signal input capture, speed
SysTick_Handler	System interrupt function (1ms): state machine, speed control
BUTTON_EXINT_IRQHandler	External IO interrupt function: START/STOP button (start/stop motor)

5) mc_type.h file

- This file contains global emulation/type definitions. Refer to Table 12 for details.

Table 12. Motor control library emulation list

Item	Description
firmware_id_type	Firmware ID type
motor_control_mode	Motor control mode (open loop control, voltage control, Id control mode, Iq control mode, speed control, torque control, position control, encoder alignment mode, etc.)
ctrl_source_type	Control source (software control, external circuit control)
process_state_type	Process state
esc_state_type	Motor control process state machine
err_code_type	Motor error type (over-voltage, under-voltage, over-temperature, over-current, encoder error, Hall error, startup error)
shunt_nbr_type	Shunt current sensing mode (1-shunt, 2-shunt or 3-shunt)
hall_learn_process_type	Hall self-learning process state
curr_offs_type	ADC current sampling offset value
current_type	Current-related structure variable
pid_ctrl_type	PID control loop related structure variable
speed_type	Speed-related structure variable
value_type	Value type
hall_sensor_type	Hall sensor related structure variable
ramp_cmd_type	Ramp command variable
usart_data_index	UART variable

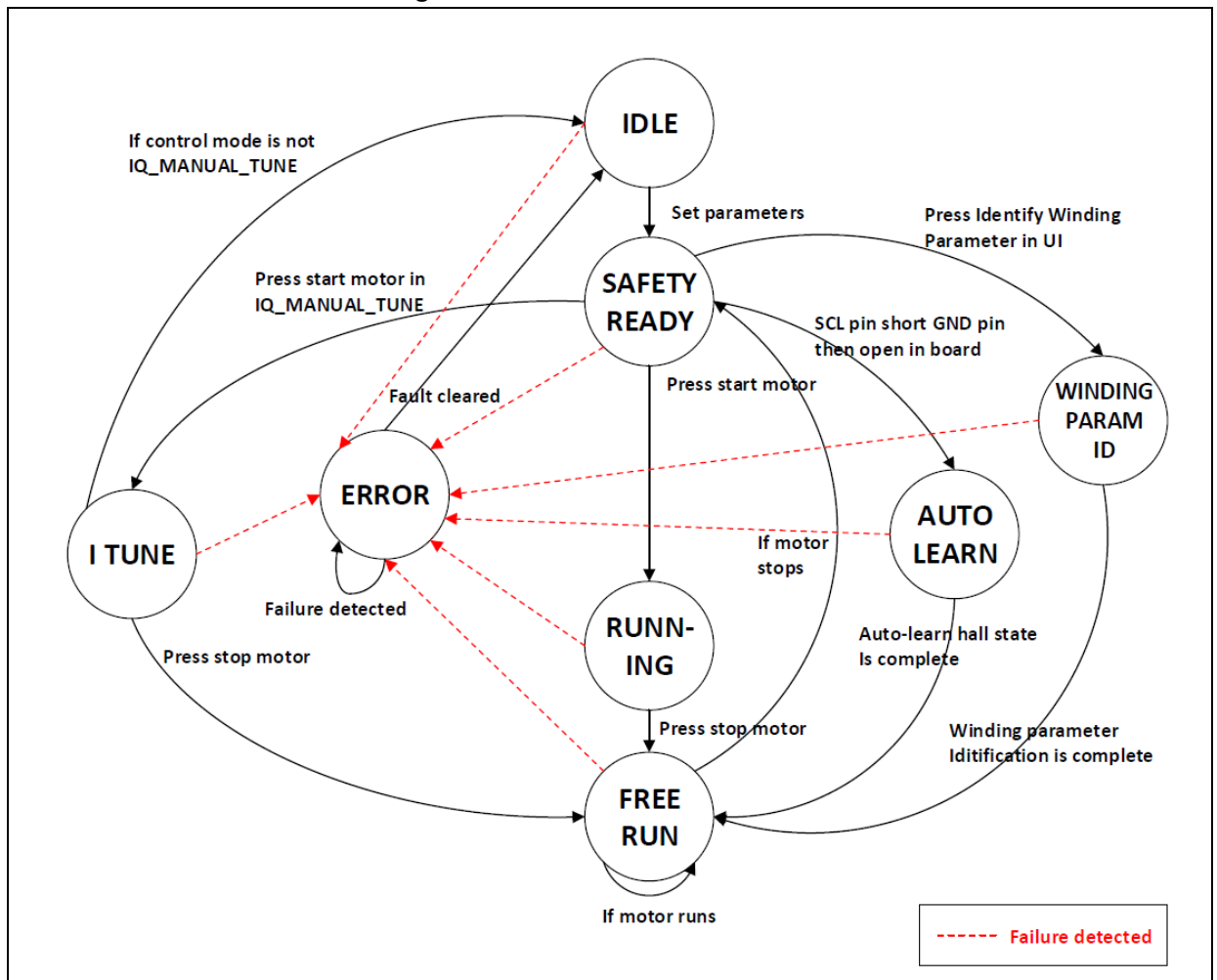
moving_average_type	Moving average related structure variable
usart_config_type	UART peripheral configuration variable
ui_wave_param_type	UI waveform display variable
i_bus_type	BUS current related structure variable
olc_type	Open loop control related structure variable
adc_sample_type	ADC sampling related variable
i_auto_tune_type	Tune current PI controller parameters
motor_param_id_type	Motor winding parameters identification
hall_learn_type	Hall self-learning variable

4 Motor control state machine

4.1 Introduction

Figure 10 illustrates the flow chart of the state machine. The initial settings of each state are checked by the Main program on a circular basis. Upon state change, the new state is programmed with initial settings. In addition, the SysTick Handler interrupt function is responsible for real-time monitoring the state machine at a millisecond rate. Once failure is detected, the state machine enters “Error” state to halt motor so as to avoid potential damage to motor or driver. The motor is being stalled until the failure is cleared.

Figure 10. State machine flow chart



The entire flow chart is composed of Idle, Safety ready, Running, Free run, I_tune and Error states.

Idle

The initial state of a state machine. Motor stays idle in this state. And the state machine returns to “Idle” state when a failure is cleared or motor stops running.

Safety ready

This state indicates that the motor can start up safely as all parameters are already set and the current offset value is available.

Running

This state indicates that the motor is running, and the user can adjust parameters (target speed, target current) or stop motor via UI interface.

Free run

This state is similar to Stop mode. In this state, the driver stops output, and the motor speed will slowly reduce to zero. This state is maintained until the motor stops running. After a full stop of motor, the state machine then returns to “Safety ready” state.

I_tune

This state is used to tune PID controller parameters. In this state, the user can tune the target current, current loop KP and KI values via UI software to generate a step current.

Winding param Id

This refer to the state of motor winding parameter identification. The user can enable this feature via UI interface. The winding parameter identification feature is often used for tuning a new motor. The tuned parameters can be used for sensorless vector control and current PI controller parameters self-tuning.

Auto learn

This refers to a hall state sequence self-learning feature in our motor control library. It can be used in six-step square wave mode and vector control mode.

Error

The state machine jumps to “Error” state in case of any failures.

Table 13 presents the initial state, end state and switch conditions between two states.

Table 13. State switch conditions

Initial state	End state	State switch condition
IDLE	SAFETY REDAY	The control source is from UI tool, and initial parameters are already set.
		The control source is from the external circuit. The POTENTIOMETER value is less than the default value, and initial parameters are already set.
SAFETY REDAY	RUNNING	When the control mode is SPEED_CTRL: If control source is from UI tool, the speed command is not 0 and click the START MOTOR (UI tool). If control source is from the external circuit, the POTENTIOMETER value is greater than the set value and press the START/STOP button on the motor control board.
		When the control mode is TORQUE_CTRL: If control source is from UI tool, the current command is not 0 and click the START MOTOR (UI tool). If control source is from the external circuit, the POTENTIOMETER value is greater than the set value and press the START/STOP button on the motor control board.
		When the control mode is OPEN_LOOP_CTRL: If control source is from UI tool, click the START MOTOR (UI tool). If control source is from the external circuit, press the START/STOP button on the motor control board..
	I TUNE	When the control mode is IQ_MANUAL_TUNE, click the START MOTOR (UI tool) or press the START/STOP button on the motor control board.
	AUTO_LEARN	Click the “Hall phase sequence self-learning” on UI tool, or the external circuit Hall self-learning pin (PB10 on AT-MOTOR-EVB board) is grounded and then released
RUNNING	WINDING_PARAM_ID	Click the “Winding parameter identification” on UI tool
	FREE_RUN	When the control mode is SPEED_CTRL or TORQUE_CTRL: If control source is from UI tool, click the STOP MOTOR (UI tool). If control source is from the external circuit, press the START/STOP button on the motor control board..
		When the control mode is OPEN_LOOP_CTRL: If control source is from UI tool, click the STOP MOTOR (UI tool). If control source is from the external circuit, press the START/STOP button on the motor control board.

FREE_RUN	SAFETY REDAY	When the actual speed is 0
ERROR	IDLE	Failure is cleared.
I TUNE	FREE_RUN	Click the STOP MOTOR (UI tool) or press the START/STOP button on the motor control board.
AUTO_LEARN	FREE_RUN	Hall state sequence self-learning is completed.
WINDING_PARAM_ID	FREE_RUN	Winding parameter identification is completed.

5 How to use demo project

For details, refer to *AN0213_AT32_BLDC_Hall_Quick_Start_Guide*.

6 Revision history

Table 14. Document revision history

Date	Version	Revision note
2023.03.02	2.0.0	Initial release.
2023.04.27	2.0.1	1. Added forward and reverse control; 2. Iterated UI interface (baud rate:1.5M)
2023.10.05	2.1.0	Modified document structure, updated parameters, and iterated UI interface.
2024.05.17	2.1.1	Added Hall phase sequence self-learning, Auto tune PI parameters, low-speed voltage control mode, parameters update.
2024.12.27	2.1.2	Added applicable products, updated state machine process flow, updated parameters and Flash ROM configuration.
2025.10.09	2.1.3	Modified Flash ROM configuration in Table 2, updated some parameters, and modified the state machine description.

IMPORTANT NOTICE – PLEASE READ CAREFULLY

Purchasers are solely responsible for the selection and use of ARTERY's products and services; ARTERY assumes no liability for purchasers' selection or use of the products and the relevant services.

No license, express or implied, to any intellectual property right is granted by ARTERY herein regardless of the existence of any previous representation in any forms. If any part of this document involves third party's products or services, it does NOT imply that ARTERY authorizes the use of the third party's products or services, or permits any of the intellectual property, or guarantees any uses of the third party's products or services or intellectual property in any way.

Except as provided in ARTERY's terms and conditions of sale for such products, ARTERY disclaims any express or implied warranty, relating to use and/or sale of the products, including but not restricted to liability or warranties relating to merchantability, fitness for a particular purpose (based on the corresponding legal situation in any unjudicial districts), or infringement of any patent, copyright, or other intellectual property right.

ARTERY's products are not designed for the following purposes, and thus not intended for the following uses: (A) Applications that have specific requirements on safety, for example: life-support applications, active implant devices, or systems that have specific requirements on product function safety; (B) Aviation applications; (C) Aerospace applications or environment; (D) Weapons, and/or (E) Other applications that may cause injuries, deaths or property damages. Since ARTERY products are not intended for the above-mentioned purposes, if purchasers apply ARTERY products to these purposes, purchasers are solely responsible for any consequences or risks caused, even if any written notice is sent to ARTERY by purchasers; in addition, purchasers are solely responsible for the compliance with all statutory and regulatory requirements regarding these uses.

Any inconsistency of the sold ARTERY products with the statement and/or technical features specification described in this document will immediately cause the invalidity of any warranty granted by ARTERY products or services stated in this document by ARTERY, and ARTERY disclaims any responsibility in any form.

© 2025 ARTERY Technology – All Rights Reserved